

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic design.

Architecture Design

Architecture Evaluation and Refinement

- ▶ Design is **iterative**: we propose design decisions, assess, make adjustments, and propose more decisions
- ▶ Many techniques to evaluate the design:
 - ▶ Trade-off analysis
 - ▶ Cost-benefit analysis

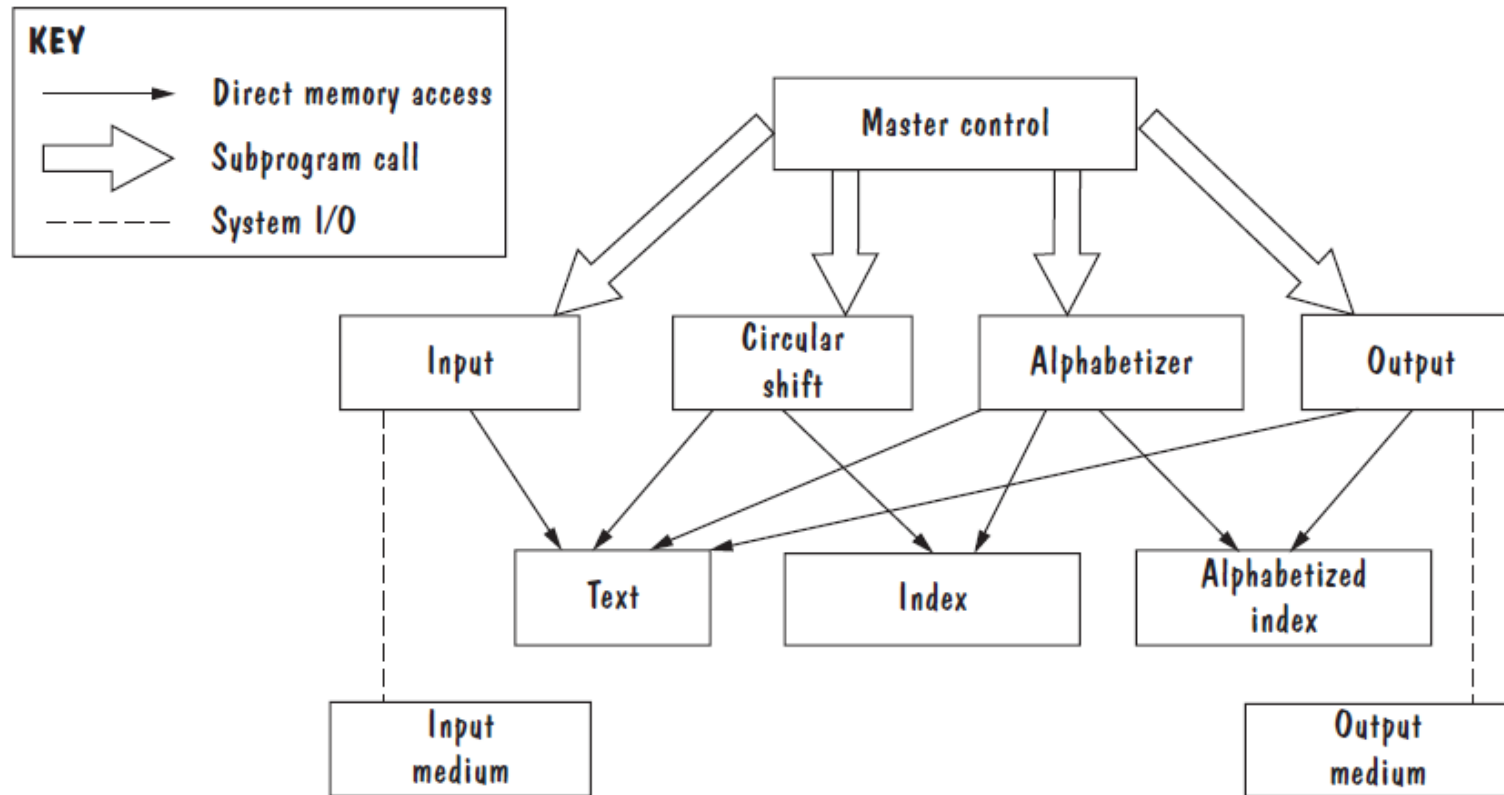
Trade-off Analysis

One Specification, Many Designs

- ▶ To see how different architecture styles can be used to solve the same problem.
- ▶ The **[key word in context] KWIC** system:
 - ▶ Accepts an ordered set of lines.
 - ▶ Each line is an ordered set of words.
 - ▶ Each word is an ordered set of characters.
 - ▶ Any line may be “circularly shifted” by repeatedly removing the first word and appending it at the end of the line.
 - ▶ The KWIC index system outputs a list of all circular shifts of all lines in alphabetical order.
- ▶ Such systems are used to **index text**, supporting rapid **searching for keywords**. For example, KWIC is used in electronic **library catalogues** and in **online help systems**.

Trade-off Analysis

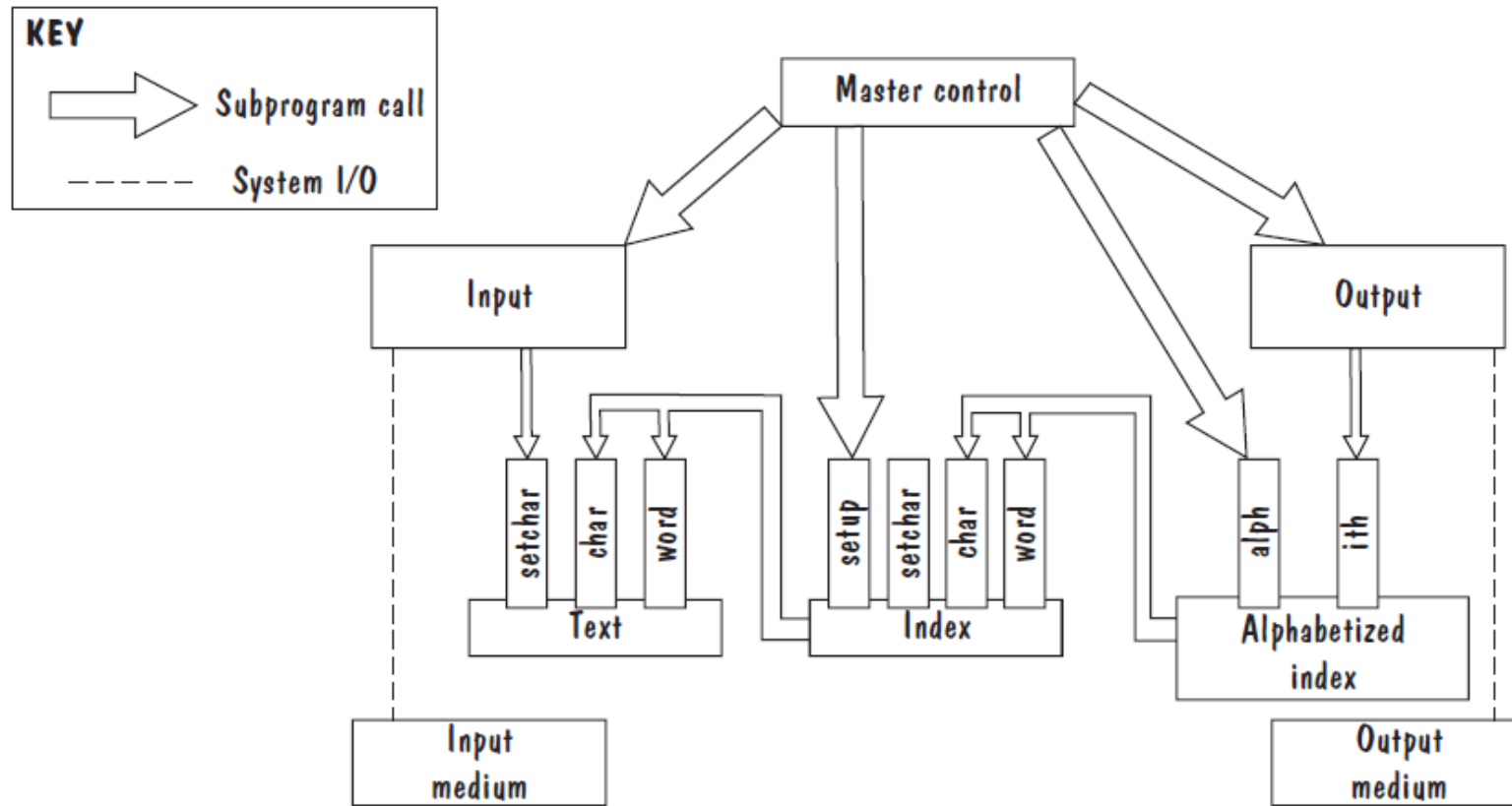
One Specification, Many Designs



Repository solution for KWIC

Trade-off Analysis

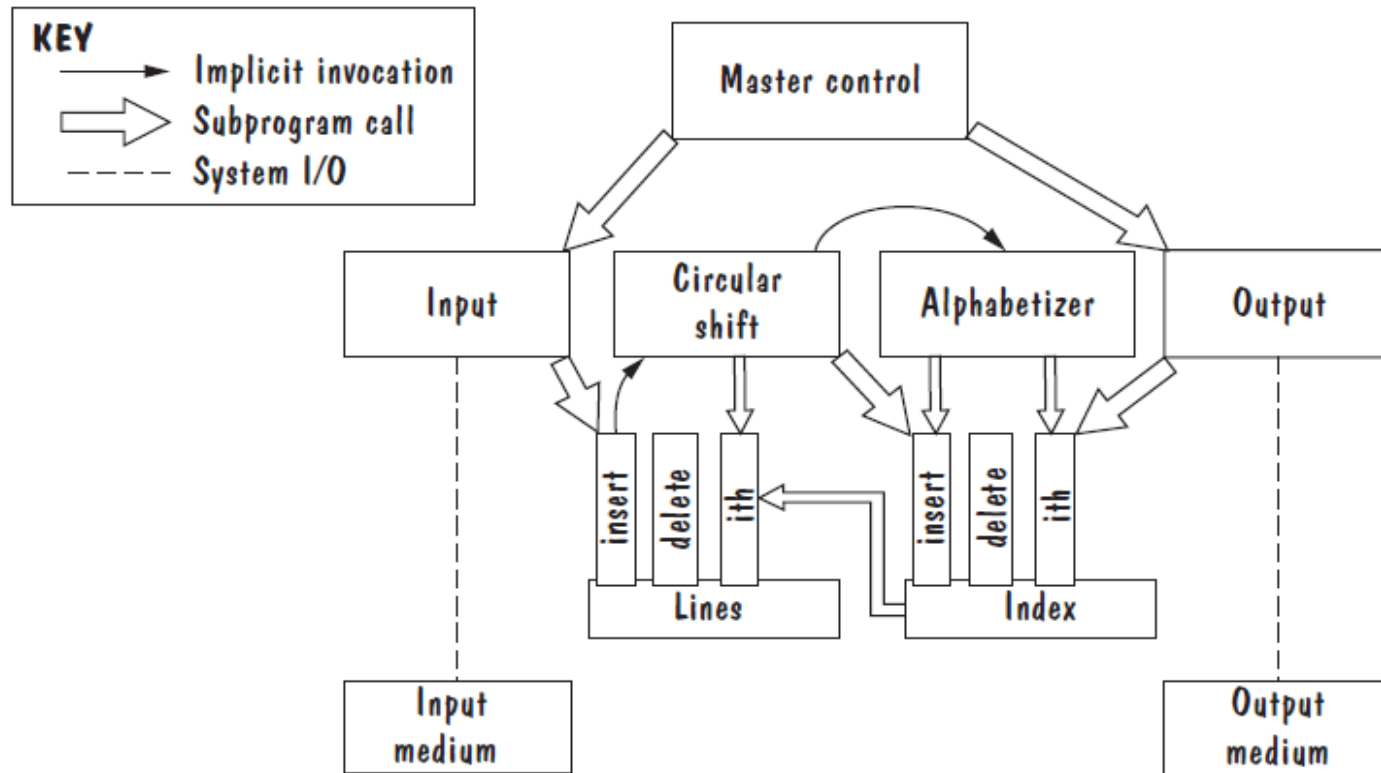
One Specification, Many Designs



Data-module solution for KWIC

Trade-off Analysis

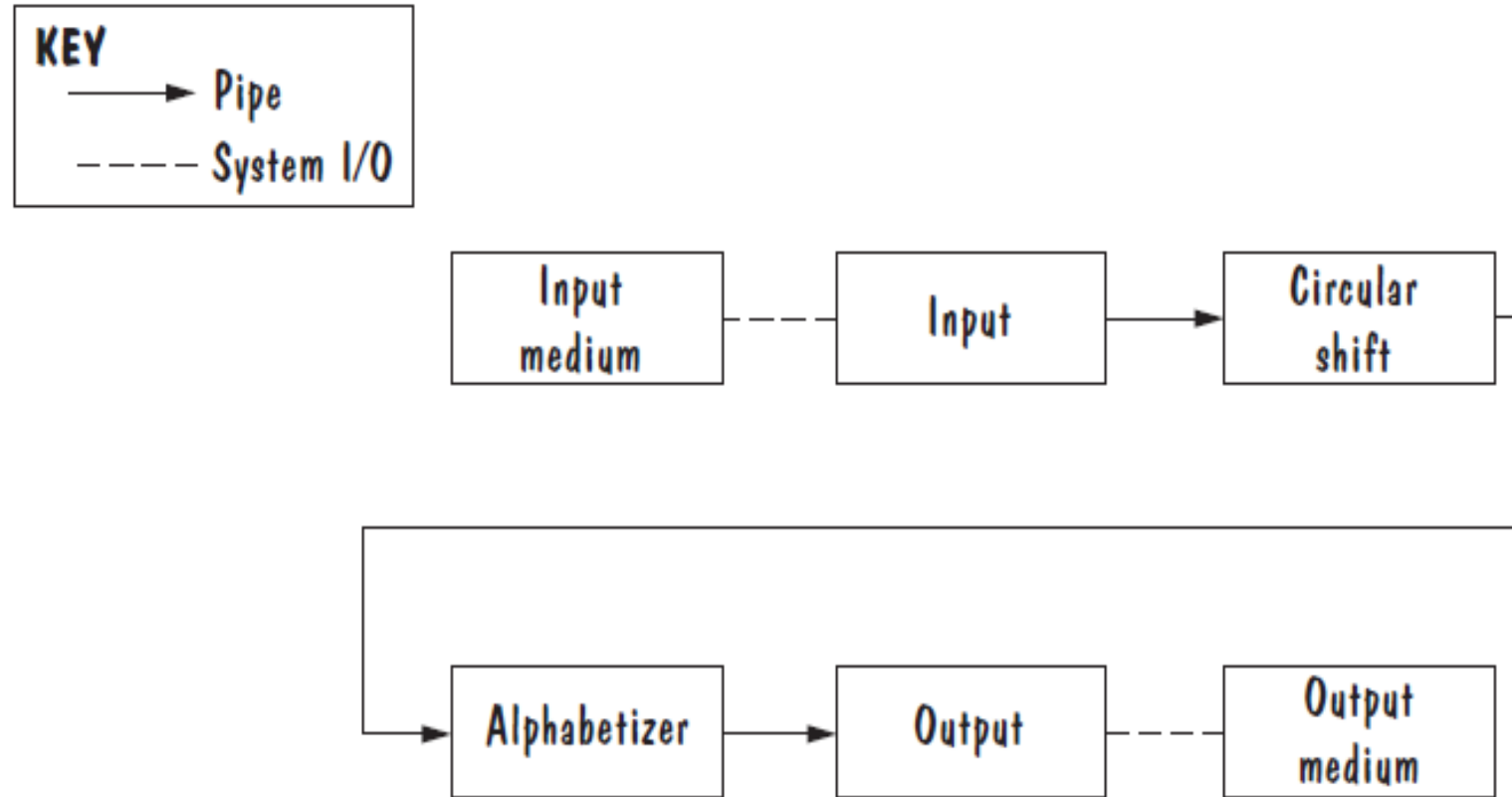
One Specification, Many Designs



ADT solution for KWIC

Trade-off Analysis

One Specification, Many Designs



Pipe-and-Filter solution for KWIC

Trade-off Analysis

One Specification, Many Designs

- ▶ Each design has its positive and negative aspects.
- ▶ Need a method for comparing different designs that allows us to choose the best one for our purpose.

Trade-off Analysis

One Specification, Many Designs (Comparison Tables)

TABLE 5.2 Comparison of Proposed KWIC Solutions

Attribute	Shared Data	Data Abstraction	Implicit Invocation	Pipe and Filter
Easy to change algorithm	—	—	+	+
Easy to change data representation	—	+	—	—
Easy to add functionality	+	—	+	+
Good performance	—	—	+	+
Efficient data representation	+	+	+	—
Easy to reuse	—	+	—	+

- A plus means that the design has the attribute
- A minus means that the design does not have the attribute

Trade-off Analysis

One Specification, Many Designs (Comparison Tables)

TABLE 5.3 Weighted Comparison of Proposed KWIC Solutions

Attribute	Priority	Shared Data	Data Abstraction	Implicit Invocation	Pipe and Filter
Easy to change algorithm	1	1	2	4	5
Easy to change data representation	4	1	5	4	1
Easy to add functionality	3	4	1	3	5
Good performance	3	4	3	3	5
Efficient data representation	3	5	5	5	1
Easy to reuse	5	1	4	5	4
Total		49	69	78	62

Highest Priority of an attribute = 5, Lowest priority = 1. For example, Reusability could be the most desirable attribute

Numbers in column 3 to 6 represent the extent to which a design satisfies the characteristic of the corresponding attribute. 1 being the lowest, 5 being the highest

Trade-off Analysis

One Specification, Many Designs (Comparison Tables)

TABLE 5.3 Weighted Comparison of Proposed KWIC Solutions

Attribute	Priority	Shared Data	Data Abstraction	Implicit Invocation	Pipe and Filter
Easy to change algorithm	1	1	2	4	5
Easy to change data representation	4	1	5	4	1
Easy to add functionality	3	4	1	3	5
Good performance	3	4	3	3	5
Efficient data representation	3	5	5	5	1
Easy to reuse	5	1	4	5	4
Total		49	69	78	62

Multiply the priorities with ratings and sum over the design to get score for each design

For example, score for pipe and filter = $1 \times 5 + 4 \times 1 + 3 \times 5 + 3 \times 5 + 3 \times 1 + 5 \times 4 = 62$.

Pick the design with the highest score. In this case, Implicit-Invocation design is chosen

Trade-off Analysis

One Specification, Many Designs

- ▶ Other attributes to consider
 - ▶ Modularity
 - ▶ Testability
 - ▶ Security
 - ▶ Ease of use
 - ▶ Ease of understanding
 - ▶ Ease of integration

Cost-Benefit Analysis

- ▶ Consider a proposal to improve KWIC performance because the number of KWIC indices have increased
 - ▶ Eliminate noise word indices?
 - ▶ Change representation of indices to bin of indices?
 - ▶ Increase server capacity by adding another computer?
- ▶ A cost-benefit analysis is a widely used business tool for estimating and comparing the costs and benefits of a proposed change

Cost-Benefit Analysis

(Computing Benefits)

Cost-Benefit Analysis

- ▶ Suppose that the current catalogue contains **70,000 entries**, and that the average size of an entry (i.e., the number of words per record, including the video's title, the director's name, the actors' names, and so on) is **70 words**, for a total of almost five million circular shifts.
- ▶ On average, it currently takes the system **0.016 seconds** to find and output all entries that contain **two keywords**, which means that the system accommodates about **60 such requests per second**.
- ▶ However, at peak times, the system will receive **up to 100 queries per second**, and the system is eventually expected to handle **200 queries per second**.

Cost-Benefit Analysis

TABLE 5.4 Cost-Benefit Analysis of Design Proposals

	Eliminate Noise Words	Store Indices in Bins	Add Second Server
Benefits			
Search time	0.015 sec	0.002 sec	0.008 sec
Throughput	72 requests/sec	500 requests/sec	115 requests/sec
Added value	\$24,000/yr	\$280,000/yr	\$110,000/yr
Costs			
Hardware			\$5,000
Software	\$50,000	\$300,000	\$200,000
Business losses	\$28,000+/yr		
Total costs first year	\$78,000	\$300,000	\$205,000

Cost-Benefit Analysis

- ▶ For simplicity, suppose that every additional request per second that the system can process, up to 200 requests/second, would save the company \$2000 per year, based on retained customers and reduced calls to technical support.
- ▶ Given this value function, eliminating noise words would save the company \$24,000 per year, calculated as:
$$(72 \text{ requests/second} - 60 \text{ requests/second}) * \$2000/\text{year} = \$24,000/\text{year}$$
- ▶ Adding a second server would save the company \$110,000 per year, calculated as:
$$(115 \text{ requests/second} - 60 \text{ requests/second}) * \$2000/\text{year} = \$110,000/\text{year}$$
- ▶ The second design option would improve the system's throughput beyond what will be needed (the system will receive at most 200 requests per second). Therefore, the value added by changing to bin-based indexing is the maximum possible value:
$$(200 \text{ requests/second} - 60 \text{ requests/second}) * \$2000/\text{year} = \$280,000/\text{year}$$

Cost-Benefit Analysis

- Payback period

$$\text{Payback Period} = \frac{\text{Initial Investment (Total Costs)}}{\text{Annual Benefit (Added Value)}}$$

- In this example, the payback period for restructuring the sorted-index module (design 2) is:

\$300,000/\$280,000 = 1.07 of a year = approximately 13 months

Trade-off Analysis (Exercise)

Attribute	Priority	Design A	Design B	Design C	Design D
Good Performance	3	3	2	5	4
High Security	5	3	5	4	4
Modularity	4	5	1	3	1
Ease of Integration	1	1	3	5	5
Easy to Reuse	2	2	1	4	3
Total		?	?	?	?

- Highest Priority of an attribute = 5, Lowest priority = 1
- Calculate the score of each design.
- Which Design will be chosen?

Trade-off Analysis (Exercise)

Attribute	Priority	Design A	Design B	Design C	Design D
Good Performance	3	3	2	5	4
High Security	5	3	5	4	4
Modularity	4	5	1	3	1
Ease of Integration	1	1	3	5	5
Easy to Reuse	2	2	1	4	3
Total		?	?	?	?

- Which Design will be chosen?
- Answer: Design C